



PRÁCTICA POO

Marcos Jiménez González

Programación Orientada a Objetos

Índice

1. Introducción	3
2. Diseño (clases y relaciones)	3
2.1 Clases principales	4
2.2 Relaciones principales	6
3. Decisiones de diseño (por qué jerarquía elegida)	7
3.1 Separación en archivos .cpp y .h	7
3.2 Jerarquía polimórfica con clase base abstracta	7
3.3 Gestión manual de memoria y propiedad explícita	7
3.4 Inclusión de Ebook como tercer tipo (Variante B1)	8
3.5 Política de sanciones simplificada (Variante A1)	8
3.6 Restricción por sanciones (Variante C3)	8
3.7 Persistencia con CSV (Variante D1)	8
3.8 Búsqueda por autor y ordenación por título (Variante E1)	8
3.9 Exportación de préstamos activos (Variante F3)	8
4. Explicación del formato de ficheros y de las operaciones por consola	9
4.1 Formato de archivos CSV	9
4.2 Funciones auxiliares	9
4.3 Operaciones por consola	9
5. Actividades realizadas por cada integrante	10

1. Introducción

El proyecto consiste en el desarrollo de un programa de C++ para la gestión de una Biblioteca Universitaria. La aplicación permite administrar items de una biblioteca, usuarios, préstamos, devoluciones, sanciones y búsquedas, además de poder guardar y cargar archivos CSV.

Las variantes seleccionadas del enunciado son: **A1, B1, C3, D1, E1 y F3.**

2. Diseño (clases y relaciones)

El sistema se compone de lo siguiente:

1. **Item.h** : declara la clase abstracta item (interfaz común para todos los tipos de materiales)
2. **Book.h/.cpp** : define e implementa la clase book (representa libros físicos y su información)
3. **Journal.h/.cpp** : declara e implementa la clase Journal (representa revistas con título y volumen)
4. **Ebook.h/.cpp** : declara e implementa la clase Ebook (incorpora licencia y fecha de expiración)
5. **User.h/.cpp** : declara e implementa la clase user (se encarga de gestionar los datos roles y sanciones de cada usuario)
6. **Loan.h/.cpp** : define e implementa la clase Loan (representa préstamo activo o devuelto entre un usuario y un item)
7. **Catalog.h/.cpp** : define e implementa la clase catalog (responsable de almacenar los items y gestionarlos (gestiona su memoria))
8. **Library.h/.cpp** : define e implementa la clase Library (coordina el funcionamiento del proyecto (catálogos, usuarios, préstamos y gestión))
9. **Utils.h/.cpp** : incluye funciones auxiliares
10. **main.cpp** : contiene la función principal del programa (menu)

2.1 Clases principales

Class Item

La clase **Item** actúa como punto de partida para todos los tipos de materiales de la biblioteca. Define la interfaz común que deben compartir libros, revistas y ebooks.

Incluye:

- Un método virtual puro `info()` const, que obliga a cada tipo de material a proporcionar su propia descripción.
- Un destructor virtual para permitir que los objetos derivados se eliminen correctamente a través de un puntero `Item*` y evitar problemas como el *object slicing*.

Class Book

La clase **Book** extiende `Item` y representa libros físicos dentro del catálogo.

Sus atributos principales son:

- title
- author

Class Journal

La clase **Journal** modela revistas o publicaciones periódicas.

Contiene:

- title
- volume

Class Ebook (Variante B1)

En esta clase se implementa la variante **B1**, correspondiente a libros electrónicos con restricciones de licencia.

Incluye:

- license
- expiryDate

Con esta clase introduce una función adicional en Library para evitar préstamos (Ebook) si la licencia está vencida.

Class User

Los usuarios del sistema cuentan con los siguientes atributos básicos:

- id
- name
- role (Student, PDI o PAS)
- sanctionAmount (importe acumulado en sanciones)
- manuallyBlocked (bloqueo aplicado por el administrador)

El diseño combina dos tipos de bloqueo:

Automático, según la variante C3, cuando las sanciones superan los 10 €.

Manual, gestionado desde el menú del programa.

Class Loan

La clase **Loan** representa un préstamo, vinculando un usuario con un material mediante punteros no propietarios. No gestiona memoria; simplemente registra la relación.

Incluye:

- User* user
- Item* item
- startDate
- dueDate
- returnDate (puede estar vacío)
- returned (marca si el ítem ha sido devuelto)

Su función es únicamente registrar el estado del préstamo sin poseer ningún recurso (solo me sirve para relacionar).

Class Catalog

Contiene un vector (std::vector items;). Es el responsable de liberar memoria de los objetos Item* cuando:

- El catálogo se destruye.
- Se elimina un material.
- Se edita un ítem y se sustituye por uno nuevo.

Se encarga de todos los delete, tanto en el destructor como al sustituir un ítem durante la edición. De esta manera no necesito smart pointers para la gestión manual de memoria.

Class Library

Clase que funciona como el “cerebro” del sistema:

Contiene:

- Un objeto Catalog que almacena los materiales.
- Vectores propietarios de User* y Loan*.

Entre sus funciones principales se encuentran:

- Añadir, editar y eliminar materiales del catálogo.
- Registrar y gestionar préstamos y devoluciones.
- Aplicar sanciones conforme a la variante A1 (0,10 € por día de retraso, con un máximo de 15 €).
- Buscar por autor y ordenar resultados por título (E1).
- Cargar y guardar datos en formato CSV (D1).
- Exportar préstamos activos en CSV (F3).
- Bloquear usuarios tanto de forma automática como manual (variante C3).

2.2 Relaciones principales:

Library

└─ le pertenece → Catalog → le pertenece Item*

└─ le pertenece → vector

└─ le pertenece → vector

3. Decisiones de diseño (por qué jerarquía elegida)

3.1 Separación en archivos .cpp y .h

Para mantener una estructura modular y fácilmente mantenible (sobre todo sin tanto código mezclado), todas las clases del proyecto se dividen entre archivos de cabecera (.h) y archivos de implementación (.cpp).

En los archivos .h se declaran los atributos, métodos y estructuras públicas necesarias para que otras clases puedan interactuar con ellas sin depender de su lógica interna.

Los archivos .cpp, contienen la implementación detallada de los métodos declarados en las cabeceras. Esta separación permite reducir dependencias, tarda menos en compilar y evita que cambios internos obliguen a recompilar todo el proyecto.

Además, el enunciado me lo pide pero no sabía si se refería a solo dos archivos (uno .cpp y otro .h) por eso he decidido hacerlo de esta manera.

3.2 Jerarquía polimórfica con clase base abstracta

Adopté una jerarquía con Item como raíz abstracta para representar distintos tipos de materiales de manera unificada. Así podía activar el polimorfismo en operaciones como info().

Entonces el código en el catálogo y en los préstamos no se duplica. Además, así añadir un nuevo tipo de material no afecta a las clases existentes.

3.3 Gestión manual de memoria y propiedad explícita

Como no podía usar smart pointers, decidí almacenar punteros crudos (Item, User, Loan*)

Para la gestión de la memoria:

- Catalog es propietario de los Item* y se encarga de liberar todos los recursos.
- Library es propietaria de usuarios y de préstamos.
- Loan utiliza punteros no propietarios para evitar destrucciones incorrectas.

Así no hay fugas de memoria y mantiene de manera clara la propiedad.

3.4 Inclusión de Ebook como tercer tipo (Variante B1)

Elegí Ebook porque lo veía como la opción más sencilla de integrar y porque me dejaba introducir lógica adicional que me parecía interesante (gestión de expiración).

3.5 Política de sanciones simplificada (Variante A1)

Elegí la variante A1 porque veía que me iba a dejar un cálculo más directo y sencillo.

$\text{sanción} = \text{días_retraso} \times 0.10 \text{ €}$ (máximo 15 €)

3.6 Restricción por sanciones (Variante C3)

Elegí C3 porque la veía como una restricción extra a los préstamos (bloqueo automático):

$\text{sanción_acumulada} > 10 \text{ €}$

3.7 Persistencia con CSV (Variante D1)

Elegí el formato CSV por su simplicidad. Dentro de lo que cabe me sonaba ya del anterior curso y el formato en Excel me parecía la mejor opción para guardar y cargar datos.

3.8 Búsqueda por autor y ordenación por título (Variante E1)

La búsqueda por autor y la ordenación alfabética por título me parecía lo más interesante porque lo veo como si se tratase de un buscador normal que te filtra los items.

3.9 Exportación de préstamos activos (Variante F3)

Como ya había estado haciendo otras cosas en el proyecto que tienen que ver con CSV quise seguir con ese mismo formato.

4. Explicación del formato de ficheros y de las operaciones por consola

4.1 Formato de archivos CSV

Se utilizan tres archivos principales: items.csv; users.csv; loans.csv.

items.csv

Su estructura es la siguiente: type,title,author,volume,license,expiry

Cada tipo de material usa solo los campos correspondientes, dejando el resto vacío.

users.csv

Su estructura es la siguiente: id,name,role,sanction

El rol se guarda como entero (Student=0, PDI=1, PAS=2).

loans.csv

Su estructura es la siguiente: userId,itemIndex,startDate,dueDate,returnDate

Un campo returnDate vacío indica préstamo activo.

4.2 Funciones auxiliares

sanearCSV()

Lo necesité para poder duplicar comillas internas y encerrar entre comillas si contiene comas.

dividirCSV()

Me sirve para reconstruir campos respetando comillas escapadas y me permite cargar de manera correcta títulos o nombres complejos.

4.3 Operaciones por consola

El interfaz se basa en un menú textual con opciones numeradas donde el usuario debe elegir qué hacer. Las opciones son:

Añadir Item	Posibilidad de añadir item a la biblioteca
Añadir Usuario	Posibilidad de añadir un usuario
Listar Items	Da el listado de items de la biblioteca
Registrar Préstamo	Asigna un item a un usuario (hacer préstamos)
Registrar Devolución	Registra un item como devuelto
Guardar Datos	Guarda los datos realizados (guarda CSV)
Cargar Datos	Carga los datos a un CSV
Buscar por autor y listado ordenado por título	Libros escritos por un autor ordenados alfabéticamente
Exportar estadísticas de préstamos activos a CSV	Genera un archivo CSV con los préstamos que no han sido devueltos, también qué usuario tiene cada item y las fechas del préstamo
Eliminar Item	Elimina un item de la biblioteca
Editar Item	Deja editar los datos de un item
Bloquear Usuario	Deja bloquear un usuario (para préstamos)
Desbloquear Usuario	Deja desbloquear un usuario
Salir	Deja salir del menú de opciones

5. Actividades realizadas por cada integrante

Como he estado yo solo, he realizado todo el proyecto.